

Algorithms:

- Sequence
- Unambiguous Statements
- Must Halt with correct O/p.
- Edge Case handled.
- Mathematical tool used for

- time analysis

- Space analysis (memory)

Must be correct

↳ Mathematical Induction

↳ Proofs.

Design

- Greedy Approach
- Divide & Conquer
- Dynamic Approach

A.

Greatest Common Divisor (GCD)

1- $\gcd(m, n) = \gcd(n, m \bmod n)$

2 - GCD small number sy bara
nae ho skta. It will always
be smaller or equal.

→ GCD will be 1 if two numbers are relatively prime no.

3. LCM. Type:

$$24: 2 \cdot 2 \cdot 2 \cdot 3$$

$$12: 2 \cdot 2 \cdot 3 \Rightarrow \text{G.C.D} = 12$$

Sieve of eratosthenes:

$$n = 25$$

→ First write counting from 1 to n.

1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25			

→ Remove the multiple of 2. → $2 \cdot 2 = 4$

→ Then remove the multiple of 3. → $3 \cdot 3 = 9$

→ Loop. $\lfloor \sqrt{n} \rfloor$ tak chaly ga.

$$\sqrt{25} = 5 \text{ tak}$$

→ We cannot remove the multiple of 4.

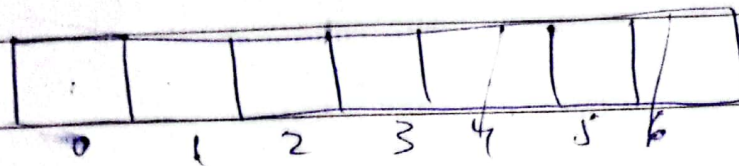
→ Then remove the multiple of 5. $5 \cdot 5 = 25$

→ 1st digit that will remove is pp. ↓

$$\text{Time} = \sqrt{n} (\log \log n)$$

$$\frac{N}{2} + \frac{N}{3} + \frac{N}{4} + \dots$$

$$N \left(\frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \dots \right) \rightarrow \text{harmonic mean}$$



Analysis Framework:

→ Space

→ time

$$T(n) = \sum_{i=1}^n C(n)$$

Basic Operation Cost

[+, -, %, *, %] Comparisons

Input (Size)



Best Case → Sorted

Worst Case → Reverse Sorted

Avg. Case → Semi-Sorted

↓
input types

Order of Growth

Efficiency Class

Linear

Exponential

Logarithmic

Cube

10

n	$O(n^2)$	$C(n^2)$
10^{20}	10^{40}	$10^{200} \rightarrow 10^{40+160} \rightarrow 10^{160} \cdot 10^{40}$
10^{40}	10^{80}	$10^{200} \rightarrow 10^{80+120} \rightarrow 10^{120} \cdot 10^{80}$
$\vdots$	$\vdots$	$\vdots$
10^{80}	10^{160}	$10^{400} \rightarrow 10^{160+240} \rightarrow 10^{240} \cdot 10^{160} \rightarrow \text{time}$

actual order of growth change name later

constant change hotyhair

insertion sort

merge sort

meeting point

→ agar chota size ho tu insertion sort or agar input size large ho tu merge sort.

Basic Operations

$x = 4 \rightarrow$ unit / constant time

$c = a + b \rightarrow$ unit time

for $i = 1$ to $n \rightarrow n$ time

$C = 1 \rightarrow n$ -time

$\rightarrow$ used for analysis

$C_{op} = 1$ if it runs in constant time.

$$T(n) = C_{op} \cdot C(n)$$

$\downarrow$ total time $\downarrow$ cost of basic operation $\downarrow$ no. of times basic operation being executed

$$T(n) \leq C(n) \text{ if } C_{op} = 1$$

$$T(n) = \frac{1}{2} n(n-1)$$

$$A = 100 \text{ ns/s}$$

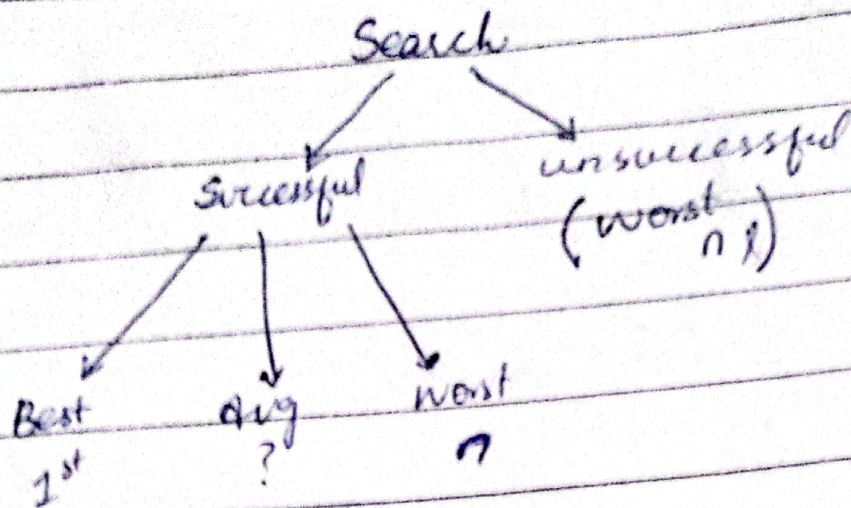
$$n = 15$$

$$B = 100,000 \text{ ns/s}$$

$$n = ?$$

$$1/1000 \Rightarrow 0.001$$

$$= 0.001 \text{ s}$$



Sorting

Insertion Sort:

Sorted $\rightarrow n$
 Reverse Sorted $\rightarrow n^2$
 Semi-sorted $\approx n^2$

Merge Sort:

Sorted = Reverse = Semi-sorted = $n \log n$

Best case $\rightarrow$ Single element array/input

Prob. that element exist = P
 does not " " = $(1-P)$

Success + Unsuccess

$$T_{avg} = \frac{P}{n} + 2\frac{P}{n} + 3\frac{P}{n} + \dots + n\frac{P}{n} + (1-P)n$$

$$T_{avg} = \frac{P}{n} (1+2+3+\dots+n) + (1-P)n$$

$$T_{avg} = \frac{P}{n} \left[\frac{n(n+1)}{2} \right] + (1-P)n$$

$$T_{avg} = \frac{P(n+1)}{2} + (1-P)n$$

$$\log a^b = b \log a$$

Date: 31/01/2025

Mon Tue Wed Thu Fri Sat

$$\text{Insertion sort} = 8n^2$$

$$\text{Merge Sort} = 64n \log n$$

Insertion sort beats merge sort

$$8n^2 \leq 64n \log n$$

$$n \leq 8 \log n$$

~~For n=1 to 5~~

$$2 \leq 8 \log 2^2$$

$$2 \leq 16$$

For n=31

$$31 \leq 8 \log 2^3$$

$$31 \leq 16$$

For n=8

$$8 \leq 8 \log 2^3$$

$$8 \leq 24$$

For n=15

$$15 \leq 8 \log 4$$

$$15 \leq 40 \quad 16 \leq 32$$

$$n = 64$$

$$n = 32$$

$$32 \leq 8 \log 2^5$$

$$32 \leq 40$$

$$64 \leq 8 \log 2^6$$

$$64 \neq 48$$

$$32 - 64$$

$$32 - 48$$

$$48 \leq 8 \log 48$$

$$48 \neq 44 \quad X$$

$$43 \leq 8 \log 43$$

$$43 \leq 43$$

$$44 \leq 8 \log 44$$

$$44 \neq 43$$

$$1 - 43$$

$$\begin{array}{r} 48 \\ 9 \times 2 \\ \hline 2 \end{array}$$

$$\log n \text{ MS} = 1s$$

$$\log n 10^{-6} s = 1s$$

$$\log n = 10^6$$

$$n = 2^{10^6}$$

Insertion Sort:

→ Decrease and Conquer

→ Space Complexity $O(n)$

→ Time Complexity $\rightarrow O(n^2)$ & $O(n)$

Cost

$$C_1 \quad \text{for } i=2 \text{ to } n \quad \checkmark \quad n+1 \quad n$$

$$C_2 \quad \text{key} = A[i] \quad n-1$$

$$O \quad // \text{insert } A[i] \text{ into sorted subarray } A[1:i-1]$$

$$C_4 \quad j = i-1 \quad n-1$$

$$C_5 \quad \checkmark \quad \text{While } j > 0 \text{ and } A[j] > \text{key} \quad (n-1) \sum_{i=2}^n t_i$$

$$C_6 \quad A[j+1] = A[j] \quad \sum_{i=2}^n (t_i - 1)$$

$$C_7 \quad j = j-1 \quad \sum_{i=2}^n (t_i - 1)$$

$$C_8 \quad A[j+1] = \text{key} \quad n-1$$

Best Case

$$T = C_1 n + C_2 (n-1) + O + C_4 (n-1) + C_5 (n-1) + C_8 (n-1)$$

$$= (C_1 + C_2 + C_4 + C_5 + C_8) n - (C_1 + C_2 + C_4 + C_5 + C_8)$$

$$= a n - b \Rightarrow O(n)$$

→ Loop Invariant (finite) is used for checking algo. ~~analysis~~ correctness.
 ↑ → i-1 tuk sorted

→ Mathematical Induction (Infinite)

Initialization
 Maintenance
 Termination

$$= C_1n + C_2(n-1) + C_4(n-1) + C_5 \left[\frac{n(n+1)}{2} - 1 \right] + C_6 \left[\frac{n(n-1)}{2} \right] + C_7 \left(\frac{n(n-1)-1}{2} \right) + C_8(n-1)$$

$$= \cancel{C_1+C_2} n^2 \left[\frac{C_5+C_6+C_7}{2} \right] + n \left[C_1+C_2+C_4+C_5 + \frac{C_8+C_5+C_6}{2} + C_7 \right] - (C_2+C_4+C_5+C_8)$$

$$= an^2 + bn - c \approx O(n^2)$$

$$100n^2$$

$$100 < 2$$

$$\frac{n(n-1)-2-1}{2}$$

Merge Sort - Inplace sorting

- Divide and Conquer
- Recursive
- Average + Worst + Best $\rightarrow n \log n$
- Loop invariant
 - ↳ correctness

- condition / Statement in algo
- proofing technique
 - Mathematical Induction
- Loop Invariant
- 1 - Initialization
 - 2 - Maintenance $\rightarrow i$ ki baat hoti
 - 3 - Termination

Insertion Sort:

- Decrease and Conquer
- Iterative / Incremental
- Best $\rightarrow$ linear n
- Worst / Average n^2 quadratic

Loop Variant:

$$(1 - i - 1)$$

- loop k around hi deikhna
- Result ki taraf lg kr jatihe.

Linear Search (Element not found jab 1 to $i-1$)

Merge Sort

Merge-Sort (A, p, r)

if $p \geq r$

return

$$q = \lfloor (p+r)/2 \rfloor$$

Merge-Sort (A, p, q)

Merge-Sort ($A, q+1, r$)

// merge $A[p:q]$ and $A[q+1:r]$ into $A[p:r]$

Merge (A, p, q, r)

Analysis:

Divide $\rightarrow$ Constant

Conquer $\rightarrow$ Recurrence $2T(n/2)$

Combine $\rightarrow n$

$$aT(n/b)$$

$a \Rightarrow$ no. of sub-problems

$b \Rightarrow$ Size of sub-problem

Divides
conquer

$$T(n) \rightarrow C(n)$$

$$\begin{array}{c} \swarrow \quad \searrow \\ T(n/2) \quad T(n/2) \end{array}$$

$$C(n)$$

$$\begin{array}{c} \swarrow \quad \searrow \\ C(n/2) \quad C(n/2) \end{array} \rightarrow$$

$$C(n)$$

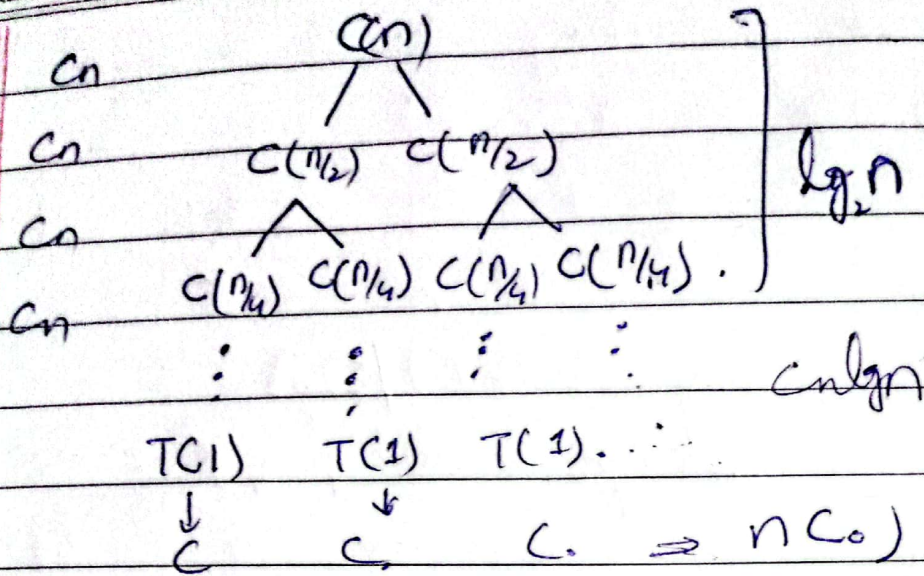
$$\begin{array}{c} \swarrow \quad \searrow \\ C(n/2) \quad C(n/2) \end{array}$$

Column 2.1, 2.3

Date: / / 20

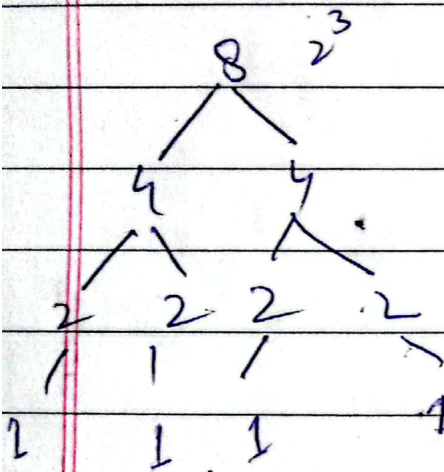
Mon Tue Wed

Mon Tue Wed Thu Fri Sat



$$lg n \rightarrow lg 2^3 \rightarrow 3$$

level = 3



$$T(n) = \begin{cases} C_0 \\ C_1 + 2T(n/2) + C_3n \\ C_1 + C_2 n lg n + C_3n \end{cases}$$

$$O(n lg n)$$

2.1.2 Le

Asymptotic Notations:

if $m \approx n$ then

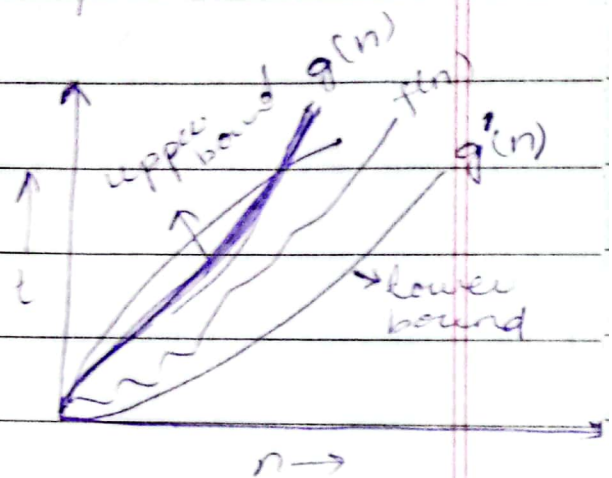
$$O = m+n$$

$$O = m+m$$

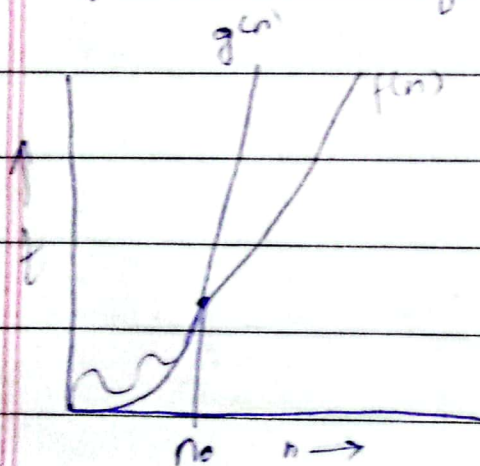
$$O = 2m \approx m$$

→ approximation / bounds.

	$O(n)$	machine $O(n)$
10	10	100
100	100	100
10000	10000	10



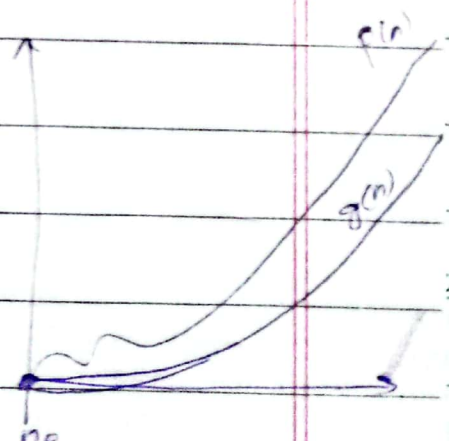
$g(n) \rightarrow$ non-function values sy draw hote.



O upper bound

(Worst

case
as $g(n)$ is increasing



Ω lower bound

(Best case)

as $g(n)$ is
increasing.

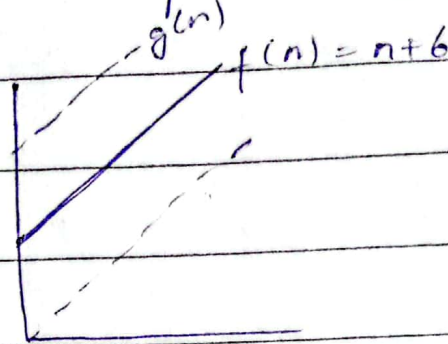
Big O:

$$f(n) \leq c g(n) \text{ for } n \geq n_0$$

Big Ω :

$$f(n) \geq c g(n) \text{ for } n \geq n_0$$

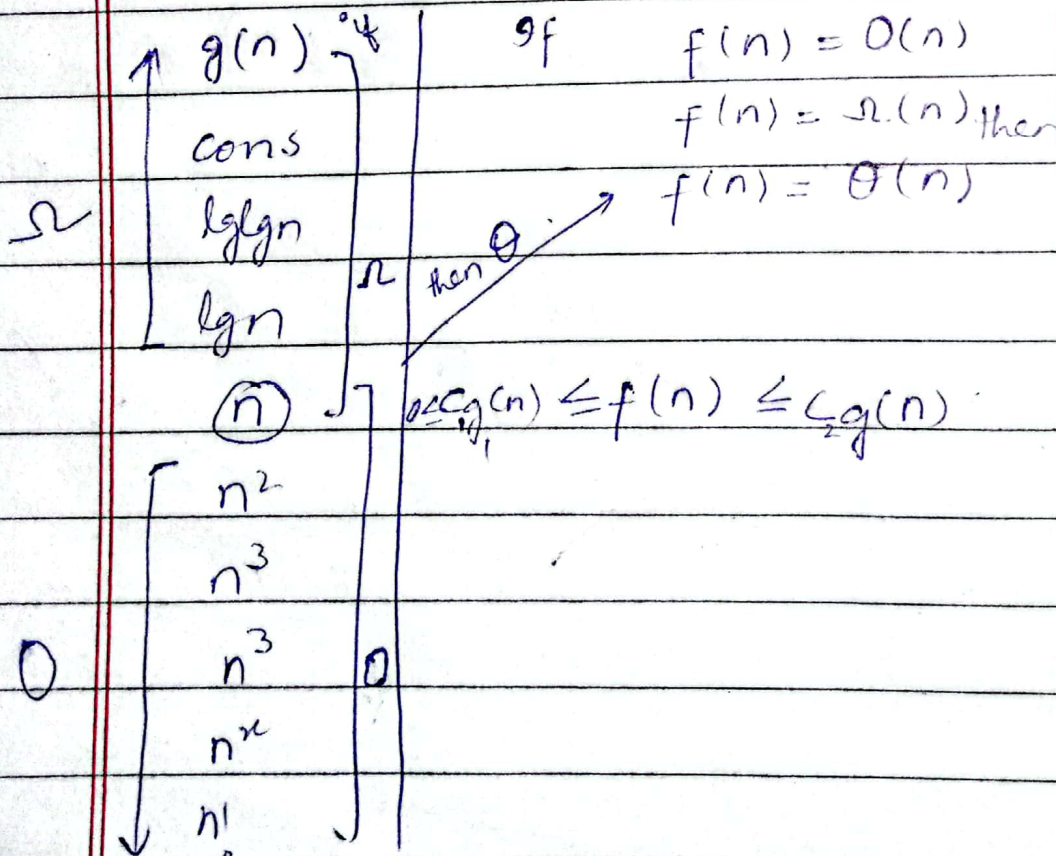
$$\uparrow f(n) \Omega g(n)$$



tight bounds
 $\left\{ \begin{array}{l} \Omega \\ O \\ \Theta \end{array} \right.$

for upper bound $O = n$ (graph)

but it is not true so we need constant



$$f(n) = n^2 + 4n + 6$$

$$n^2 + 4n + 6 \in \Theta(n^2)$$

$$\left\{ \begin{array}{l} \rightarrow n^2 + 4n + 6 = O(n^2) \\ \rightarrow n^2 + 4n + 6 = \Omega(n^2) \end{array} \right.$$

constant can be fractional but can't be positive negative

Upper bound $n^2 + 4n + 6 \leq Cn^2$

$$n^2 + 4n + 6 \leq 2n^2$$

$$n=1$$

$$n=2$$

$$n=5$$

$$n=10$$

Lower bound $n^2 + 4n + 6 \geq \frac{1}{2}n^2$

L.B. $n^2 + 4n + 6 \geq n$

$$2n^2 + 3n + 4 \in O(n^2) \quad n > 1 ; n \geq n_0$$

$$2n^2 + 3n + 4 \leq 2n^2 + 3n^2 + 4n^2$$

$$\leq 9n^2$$

constant

$$\leq c g(n)$$

$$n_0 = 1$$

$$2n^2 + 3n + 4 \in O(n^4).$$

$$\lg_4(2n^2 + 3n + 4) \in O(\lg_4 n)$$

$$\lg_4(2n^2 + 3n + 4) \leq \lg_4(2n^2 + 3n^2 + 4n^2)$$

$$3n \leq 3n^2$$

$$1 \leq n$$

$$4 \leq 4n^2$$

$$1 \leq n^2$$

$$1 \leq n$$

$$\leq \lg_4(9n^2) \quad \text{for } n \geq 1$$

$$\leq \log_4 9 + \log_4 n^2$$

$$\leq \log_4 9 + 2\log_4 n \quad \text{for } n \geq 9$$

$$\leq 3\log_4 n$$

$$\log_4 9 \leq \log_4 n$$

$$4^{\log_4 9} \leq 4^{\log_4 n}$$

$$9 \leq n$$

$$\lg_4(6n^2+9n+4) \in O(\lg_4 n)$$

$$\begin{aligned} \lg_4(6n^2+9n+4) &\leq \lg_4(6n^2+9n^2+4n^2) \\ &\leq \lg_4(19n^2) \quad n \geq 1 \\ &\leq \lg_4 19 + \lg_4 n^2 \\ &\leq \lg_4 n + 2\lg_4 n \quad \text{for } n \geq 19 \\ &\leq 3\lg_4 n \end{aligned}$$

$$\begin{aligned} \lg_4 19 &\leq \lg_4 n \\ \therefore \lg_4 19 &\leq \lg_4 n \\ 19 &\leq n \end{aligned}$$

$$\log_4(2n^2-6n+4) \in O(\log_4 n)$$

If there is any negative value in expression then remove it.

$$\begin{aligned} \log_4(2n^2-6n+4) &\leq \log_4(2n^2-\cancel{6n}+4) \\ &\leq \log_4(2n^2+4n^2) \end{aligned}$$

$$\begin{aligned} \log_4 6 &\leq \log_4 n \\ \therefore \log_4 6 &\leq \log_4 n \\ &\leq \log_4(6n^2) \\ &\leq \log_4 6 + \log_4 n^2 \quad \checkmark \\ &\leq \log_4 n + 2\log_4 n \quad \text{for } n \geq 6 \\ &= \boxed{3\log_4 n} \end{aligned}$$

$$\rightarrow 2n^2 + 3n + 4 \in \Omega(n^2)$$

$$2n^2 + 3n + 4 \geq n^2$$

~~$$2n^2 + 3n + 4 \geq 2n^2 + 3n^2 + 4n^2$$~~

$$2n^2 + 3n + 4 \geq 2n^2 \text{ or } n^2$$

$$\rightarrow 2n^2 + 3n + 4 \in \Omega(n)$$

$$2n^2 + 3n + 4 \geq n$$

$$3n \geq n$$

$$\rightarrow 2n^2 - 3n + 4 \in \Omega(n^2)$$

$$2n^2 - 3n + 4 \geq n(n^2)$$

$$2n^2 - 3n + 4 \geq 2n^2 - 3n^2$$

take smaller -ve

if you don't ignore

$$2n^2 - 3n + 4 \geq n^2$$

$$2n^2 - 3n + 4 \geq 2n^2 - 3n^2 + 4n^2$$

$$2n^2 - 3n + 4 \geq 3n^2$$

$$\sqrt{6n^3 + 7n^2 + 2n + 6} = O(n^{1.5})$$

$$\sqrt{6n^3 + 7n^2 + 2n + 6} = O(n^{1.5})$$

$$\sqrt{6n^3 + 7n^2 + 2n + 6} = O(n^{1.5})$$

$$\sqrt{6n^3 + 7n^2 + 2n + 6} = O(n^{1.5})$$

$$\sqrt{6n^3 + 7n^2 + 2n + 6} \leq \sqrt{6n^3 + 7n^3 + 2n^3 + 6n^3}$$

$$\leq \sqrt{21n^3}$$

$$\leq \sqrt{21} n^{1.5}$$

For omega:

$$\sqrt{6n^3 + 7n^2 + 2n + 6} = \Omega(n^{1.5})$$

$$\sqrt{6n^3 + 7n^2 + 2n + 6} \geq n^{1.5}$$

$$\sqrt{6n^3} \geq n^{1.5}$$

$$\sqrt{6} n^{1.5} \geq n^{1.5}$$

Analysis Framework:

Max-Value(A, n)

maxValue = A[0]

for i = 1 → n-1

if A[i] > MaxValue

Max-value = A[i]

return max-value.

→ Iterative Code

→ Recursive code

Using basic operation

$$\sum_{l=0}^u 1 = u - l + 1$$

$$\sum_{i=1}^{n-1} 1 = n - 1 - 1 + 1 \Rightarrow n - 1 \Rightarrow O(n) \rightarrow O(n^2)$$

UniqueValue(A, n)

for i = 0 → n-2

for j = i+1 → n-1 do

if A[i] == A[j] return false

return true.

~~n-1-i~~ ~~A[i]~~

$$\sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = u - l + 1 \Rightarrow \sum_{i=0}^{n-2} (n - i - 1)$$

$$= \sum_{i=0}^{n-2} (n-1-i)$$

$$= \sum_{i=0}^{n-2} (n-1) - \sum_{i=0}^{n-2} i$$

$\because n-1$ is constant

$$= (n-1) \sum_{i=0}^{n-2} 1 - \sum_{i=0}^{n-2} i$$

$$= (n-1)(n-2-0+1) - \frac{n(n+1)}{2} \quad \begin{matrix} \nearrow \text{Sum} \\ = n-2 \end{matrix}$$

$$(n-1)(n-1) - \frac{(n-2)(n-1)}{2}$$

$$= n^2 - 2n + 1 - \frac{n^2 - 3n + 2}{2}$$

$$= \frac{2n^2 - 4n + 2 - n^2 + 3n - 2}{2}$$

$$= \frac{n^2 - n}{2} \Rightarrow \frac{1}{2} (n^2 - n) \quad \begin{matrix} \hookrightarrow O(n^2) \\ \Omega(n^2) \end{matrix}$$

Matrix Mul (A, B)

for $i=0 \rightarrow n-1$

for $j=0 \rightarrow n-1$

$C[i, j] = [0, 0]$

for $k=0 \rightarrow n-1$

$C[i, j] = C[i, j] + A[i, k] + B[k, j]$

return $C[i, j]$

$$\sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1$$

$0-l+1$

$$= \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} (n-1-0+1)$$

$$= \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} n$$

$$= n \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} 1$$

$$= n \sum_{i=0}^{n-1} (n-1-0+1)$$

=

$$= n \sum_{i=0}^{n-1} n \Rightarrow n^2 \sum_{i=0}^{n-1} 1$$

$$= n^2 (n-1-0+1)$$

$$= n^2(n) = n^3$$

==o

Recurrence

1'

factorial (n)

if $n=0$ return 1

else return factorial(n-1)*n

14x5 fact(5)

6x4 fact(4)

3x2 fact(3)

2x1 fact(2)

1x1 fact(1) → 1

fact(0) → 0

fact(2) = fact(1) * n

multiplication

$M(n) = M(n-1) + 1$

$M(0) = 0$

Backward Substitution:

$$M(n) = M(n-1) + 1 \rightarrow \text{put}$$

$$= (M(n-2) + 1) + 1$$

$$= (M(n-3) + 1) + 1 + 1$$

$$= (M(n-4) + 1) + 1 + 1 + 1$$

$\vdots$

$$= M(n-i) + i$$

if for i

$$n-i = 0 \\ n = i$$

$$= M(n-n) + n$$

$$= M(0) + n \Rightarrow 1 + n$$

Tower of Hanoi:

$$M(n) = M(n-1) + 1 + M(n-1)$$

$$= 2M(n-1) + 1$$

$$= 2[2M(n-2) + 1] + 1$$

$$= [4M(n-2) + 2] + 1$$

$$= 4[2M(n-3) + 1] + 2 + 1$$

$$= 8M(n-3) + 4 + 2 + 1$$

$$= 8[2M(n-4) + 1] + 4 + 2 + 1$$

$$= 16M(n-4) + 8 + 4 + 2 + 1$$

$$2^i M(n-i) + 2^i - 1$$

$$= \quad n-i = 1$$

$$2^{n-1} M(n-1+1) + 2^{n-1} - 1 \quad n-1 = i$$

$$M(1) = 1$$

Date: ___/___/20___

Mon Tue Wed Thu Fri Sat

$$= 2^{n-1} M(1) + 2^{n-1} - 1$$

$$= 2^{n-1} + 2^{n-1} - 1$$

$$= 2 \cdot 2^{n-1} - 1$$

$$= 2^{n-1+1} - 1$$

$$= 2^n - 1 \Rightarrow 2^n \Rightarrow O(2^n)$$